

Факультет «Информатика и системы управления»
Кафедра ИУ5 «Системы обработки информации и управления»

Курс «Парадигмы и конструкции языков программирования»

Отчет по рубежному контролю №2

«Вариант А, 11»

Выполнил:

Студент группы ИУ5-31Б

Никоноров Денис

Проверил:

Гапанюк Ю.Е.

2025 г.

Листинг кода

main.py

```
from operator import itemgetter

class User:
    def __init__(self, id, fio, sal, comp_id):
        self.id = id
        self.fio = fio
        self.sal = sal
        self.comp_id = comp_id

class Computer:
    def __init__(self, id, name):
        self.id = id
        self.name = name

class UserComputer:
    def __init__(self, comp_id, user_id):
        self.comp_id = comp_id
        self.user_id = user_id

class DataProcessor:
    def __init__(self, computers, users, users_computers):
        self.computers = computers
        self.users = users
        self.users_computers = users_computers

    def get_one_to_many(self):
        return [(u.fio, u.sal, c.name)
                for c in self.computers
                for u in self.users
                if u.comp_id == c.id]

    def get_many_to_many(self):
        many_to_many_temp = [(c.name, uc.comp_id, uc.user_id)
                              for c in self.computers
                              for uc in self.users_computers
                              if c.id == uc.comp_id]

        return [(u.fio, u.sal, comp_name)
                for comp_name, comp_id, user_id in many_to_many_temp
                for u in self.users if u.id == user_id]

    def task_a1(self):
        one_to_many = self.get_one_to_many()
        return sorted(one_to_many, key=itemgetter(2))

    def task_a2(self):
```

```

one_to_many = self.get_one_to_many()
result = []

for c in self.computers:
    c_users = list(filter(lambda i: i[2] == c.name, one_to_many))
    if len(c_users) > 0:
        c_sals = [sal for _, sal, _ in c_users]
        c_sals_sum = sum(c_sals)
        result.append((c.name, c_sals_sum))

return sorted(result, key=itemgetter(1), reverse=True)

def task_a3(self):
    many_to_many = self.get_many_to_many()
    result = []

    for c in self.computers:
        if 'компьютер' in c.name:
            c_users = list(filter(lambda i: i[2] == c.name, many_to_many))
            c_users_names = [x for x, _, _ in c_users]
            for user_name in c_users_names:
                result.append((c.name, user_name))

    return result

def create_test_data():
    computers = [
        Computer(1, 'компьютер отдела по борьбе с коррупцией'),
        Computer(2, 'архивный компьютер'),
        Computer(3, 'бухгалтерский компьютер'),
        Computer(11, 'игровой компьютер отдела кадров'),
        Computer(22, 'компьютер учета взяток'),
        Computer(33, 'компьютер главного бездельника'),
    ]

    users = [
        User(1, 'Сидоров', 25000, 1),
        User(2, 'Петров', 35000, 2),
        User(3, 'Яковлев', 45000, 3),
        User(4, 'Иванов', 35000, 3),
        User(5, 'Грелкин', 25000, 3),
    ]

    users_computers = [
        UserComputer(1, 1),
        UserComputer(2, 2),
        UserComputer(3, 3),
        UserComputer(3, 4),
        UserComputer(3, 5),
        UserComputer(11, 1),
    ]

```

```

        UserComputer(22, 2),
        UserComputer(33, 3),
        UserComputer(33, 4),
        UserComputer(33, 5),
    ]

    return computers, users, users_computers

def print_table(headers, data):
    if not data:
        return

    col_widths = []
    for i in range(len(headers)):
        col_width = max(len(str(headers[i])),
                        max(len(str(row[i])) for row in data))
        col_widths.append(col_width)

    header_row = " ".join(f"{headers[i]:<{col_widths[i]}}" for i in
range(len(headers)))
    print(header_row)

    separator = " ".join("-" * col_widths[i] for i in range(len(headers)))
    print(separator)

    for row in data:
        data_row = " ".join(f"{str(row[i]):<{col_widths[i]}}" for i in
range(len(row)))
        print(data_row)
    print()

def main():
    computers, users, users_computers = create_test_data()
    processor = DataProcessor(computers, users, users_computers)

    print('Задание A1')
    print('Список всех связанных пользователей и компьютеров (сортировка по
компьютерам)')
    result_a1 = processor.task_a1()
    print_table(['Фамилия', 'Зарплата', 'Компьютер'], result_a1)

    print('Задание A2')
    print('Список компьютеров с суммарной зарплатой пользователей')
    result_a2 = processor.task_a2()
    print_table(['Компьютер', 'Суммарная зарплата'], result_a2)

    print('Задание A3')
    print('Компьютеры с названием содержащим "компьютер" и их пользователи')
    result_a3 = processor.task_a3()

```

```
print_table(['Компьютер', 'Пользователь'], result_a3)
```

```
if __name__ == '__main__':  
    main()
```

test_main.py

```
import unittest  
from main import User, Computer, UserComputer, DataProcessor, create_test_data
```

```
class TestDataProcessor(unittest.TestCase):  
    def setUp(self):  
        computers, users, users_computers = create_test_data()  
        self.processor = DataProcessor(computers, users, users_computers)  
  
    def test_task_a1_structure(self):  
        result = self.processor.task_a1()  
  
        self.assertGreater(len(result), 0)  
  
        for item in result:  
            self.assertEqual(len(item), 3)  
            self.assertIsInstance(item[0], str)  
            self.assertIsInstance(item[1], int)  
            self.assertIsInstance(item[2], str)  
  
    def test_task_a2_calculation(self):  
        result = self.processor.task_a2()  
  
        self.assertGreater(len(result), 0)  
  
        for item in result:  
            self.assertEqual(len(item), 2)  
            self.assertIsInstance(item[0], str)  
            self.assertIsInstance(item[1], int)  
  
        for i in range(len(result) - 1):  
            self.assertGreaterEqual(result[i][1], result[i + 1][1])  
  
    def test_task_a3_filtering(self):  
        result = self.processor.task_a3()  
  
        for item in result:  
            self.assertEqual(len(item), 2)  
            self.assertIsInstance(item[0], str)  
            self.assertIsInstance(item[1], str)  
            self.assertIn('компьютер', item[0].lower())  
  
    def test_one_to_many_relationship(self):
```

```

        one_to_many = self.processor.get_one_to_many()

        for fio, sal, comp_name in one_to_many:
            computer = next((c for c in self.processor.computers if c.name ==
comp_name), None)
            self.assertIsNotNone(computer)

            user = next((u for u in self.processor.users if u.fio == fio and
u.sal == sal), None)
            self.assertIsNotNone(user)

            self.assertEqual(user.comp_id, computer.id)

    def test_many_to_many_relationship(self):
        many_to_many = self.processor.get_many_to_many()

        for fio, sal, comp_name in many_to_many:
            computer = next((c for c in self.processor.computers if c.name ==
comp_name), None)
            self.assertIsNotNone(computer)

            user = next((u for u in self.processor.users if u.fio == fio and
u.sal == sal), None)
            self.assertIsNotNone(user)

            connection = next((uc for uc in self.processor.users_computers
if uc.comp_id == computer.id and uc.user_id ==
user.id), None)
            self.assertIsNotNone(connection)

class TestDataClasses(unittest.TestCase):
    def test_user_creation(self):
        user = User(1, 'Иванов', 30000, 1)
        self.assertEqual(user.id, 1)
        self.assertEqual(user.fio, 'Иванов')
        self.assertEqual(user.sal, 30000)
        self.assertEqual(user.comp_id, 1)

    def test_computer_creation(self):
        computer = Computer(1, 'компьютер отдела по борьбе с коррупцией')
        self.assertEqual(computer.id, 1)
        self.assertEqual(computer.name, 'компьютер отдела по борьбе с
коррупцией')

    def test_user_computer_creation(self):
        user_computer = UserComputer(1, 2)
        self.assertEqual(user_computer.comp_id, 1)
        self.assertEqual(user_computer.user_id, 2)

```

```
def run_tests():
    loader = unittest.TestLoader()
    suite = unittest.TestSuite()

    suite.addTests(loader.loadTestsFromTestCase(TestDataProcessor))
    suite.addTests(loader.loadTestsFromTestCase(TestDataClasses))

    runner = unittest.TextTestRunner(verbosity=2)
    result = runner.run(suite)

    return result

if __name__ == '__main__':
    run_tests()
```

run_tests.py

```
import unittest
from test_main import TestDataProcessor, TestDataClasses, run_tests

if __name__ == '__main__':
    result = run_tests()

    print("=" * 60)
    print("Результаты тестирования:")
    print(f"Всего тестов: {result.testsRun}")
    print(f"Успешно: {result.testsRun - len(result.failures)} -
len(result.errors)}")
    print(f"Провалено: {len(result.failures)}")
    print(f"Ошибок: {len(result.errors)}")
    print("=" * 60)
```

Результат выполнения

```
● nikod@LAPTOP-752EM865:~/bmstu/Nikonorov_Labs_2025$ python3 run_tests.py
test_many_to_many_relationship (test_main.TestDataProcessor.test_many_to_many_relationship) ... ok
test_one_to_many_relationship (test_main.TestDataProcessor.test_one_to_many_relationship) ... ok
test_task_a1_structure (test_main.TestDataProcessor.test_task_a1_structure) ... ok
test_task_a2_calculation (test_main.TestDataProcessor.test_task_a2_calculation) ... ok
test_task_a3_filtering (test_main.TestDataProcessor.test_task_a3_filtering) ... ok
test_computer_creation (test_main.TestDataClasses.test_computer_creation) ... ok
test_user_computer_creation (test_main.TestDataClasses.test_user_computer_creation) ... ok
test_user_creation (test_main.TestDataClasses.test_user_creation) ... ok

-----
Ran 8 tests in 0.002s

OK
=====
Результаты тестирования:
Всего тестов: 8
Успешно: 8
Провалено: 0
Ошибок: 0
=====
```